2017-2018

Classification of cancer types from medical stained images

Semester project

Wenqi Shu-Quartier-dit-Maire
Supervised by Pr Pascal Frossard
Mireille El Gheche
Bastien Pasdeloup

Summary

1. Introduction

1.1. Machine learning: healthcare applications

In recent years, there has been a dramatic increase in the use of computational methods within the medical field. Indeed, the digital revolution has provided relatively inexpensive and available means to collect and store data, whereas machine learning algorithms have enabled computers to “learn” the relevant features of a given dataset and to make predictions. The enormous amount of data generated by healthcare nowadays pairs up perfectly with the machine learning systems which can improve themselves through data experience.

Fields as diverse as disease identification, personalized treatment, drug discovery and epidemic outbreak prediction are being improved thanks to machine learning. However, it is in image-based diagnosis, disease prognosis and risk assessment that the machine learning approaches are mainly successful.

1. 2. Why use tools provided by survival analysis?

Traditionally, survival analysis was developed to measure lifespans of individuals. Its algorithms would be used to answer questions like: “how long does this population live for?”, the population being a nation’s population or a population stricken by a disease, etc. The difference between survival analysis tools and classical machine learning tools is in the fact that sometimes, at the time we want to make inferences about durations, it is possible that not all the death events have occurred yet. For example, if we want to use classical machine learning methods to predict the average lifespan of the population of a study, we would have to wait perhaps 50 years for each individual in the study to pass away before investigating - the medical professionals would rather be interested in the effectiveness of improving lifetimes after only a few years, or months possibly.

So, survival analysis allows us to handle right-censored data, i.e. individuals in a population who have not been subject to the death event or for whom the rest of the life history cannot be seen due to some external circumstances. For these individuals, instead of ignoring them (as we have to do if we use classical machine learning algorithms), survival analysis uses their current lifetime durations and tries to take advantage of this information. [1]

There are two fundamental functions:

➔ ***The survival function***

Let T be the random variable of the lifespan of an individual, and t be a random lifetime taken from the population under study. The survival function $S(t)$ of the individual is defined as the probability that the death event has not occurred yet at time T or, equivalently, as the probability of surviving past time t:

$$S(t) = \Pr(T > t)$$

➔ ***The hazard function***

The hazard function at time t is the probability of the death event occurring at time t, given that the death event has not occurred until time t:

$$\lambda(t) = \lim_{\delta t \to 0} \frac{\Pr(t \leq T \leq t + \delta t | T > t)}{\delta t}$$

It can be noticed that if we can switch from a function to another quite easily:

$$\lambda(t) = -\frac{S'(t)}{S(t)}; \qquad S(t) = \exp\left(-\int_0^t \lambda(z)dz\right)$$

Now that we have the mathematical objects on which survival analysis relies, the aim is to estimate these functions.

- **Kaplan-Meier estimator**

To estimate the survival function non-parametrically (i.e. without assuming anything about how the survival curve looks) from the data, we can use the Kaplan-Meier estimate, defined as follows:

$$\hat{S}(t) = \prod_{t_i < t} \frac{n_i - d_i}{n_i}$$

Where d_i is the number of death events at time t and n_i the number of subjects at risk of death just prior to time t. [2]

```
from lifelines import KaplaMeierFitter
duration_train = durations_subjects_of_the_training_set_were_observed_for
event_train = whether_subjects_of_the_training_set_are_dead_or_not
model = KaplanMeierFitter()
model.fit(durations=duration_train, event_observed=event_train)
# plot the survival function and its confidence intervals
model.plot()
# calculate the survival probability for time t: S(t)
model.predict(t)
```

- **Weibull model**

Another very popular model for survival data is the Weibull model [3]. Unlike the Kaplan-Meier estimate, this model is parametric (i.e. has a functional form with parameters that we fit the data to):

$$\hat{S}(t) = e^{-(\lambda.t)^{\rho}} \qquad \rho > 0;\ \lambda > 0$$

```
from lifelines import WeibullFitter
model = WeibullFitter()
model.fit(duration_train, event_train)
print(model.lambda_, model.rho_)
```

- **Nelson-Aalen estimator**

If we are curious about the hazard function $\lambda(t)$ of a population, we cannot transform the Kaplan-Meier estimate but we can estimate the *cumulative* hazard function $\Lambda(t) = \int_0^t \lambda(z)dz$ with the Nelson-Aalen [4] estimator:

$$\widehat{\Lambda}(t) = \sum_{t_i \leq t} \frac{d_i}{n_i}$$

Where d_i is the number of deaths at time t_i, and n_i is the total number of individuals at risk at t_i. We are estimating cumulative hazard curve $\Lambda(t)$ and not hazard curve $\lambda(t)$ because the sum of estimates is much more stable than the point-wise estimates. [5]

```
from lifelines import NelsonAalenFitter
model = NelsonAalenFitter()
model.fit(duration_train, event_train)
model.plot()
```

Yet most of the time we also have additional data aside from the duration and the censorships: we can use survival regression [6] techniques to regress covariates (e.g. stage of the disease, age, gender, etc.) against the durations and lifetimes. There are two popular competing techniques in survival regression: Cox's model and Aalen's additive model. Both models attempt to represent the hazard rate $\lambda(t|x)$ as a function of t and some covariates regrouped in the vector x.

- **Cox's Proportional Hazard model**

The idea behind the model is that the log-hazard of an individual is a linear function of their static covariates and a population-level baseline hazard that changes over time:

$$\lambda(t|x) = b_0(t).\exp\left(\sum_{i=1}^{d} b_i x_i\right)$$

There is something to notice about this model: the only time component is in the baseline hazard, $b_0(t)$. The term in exp is only a scalar factor that only decreases or increases the baseline hazard; hence a change in a covariate will only increase or decrease this baseline hazard. It is a very strong assumption that has to be verified.

After fitting, in order to see the goodness of fit of the model to the data, we can get the score of the model or compare the spread between the baseline survival function and the Kaplan-Meier survival function. Indeed, a small spread between these two curves means that the impact of the exponential in the Cox model does very little, whereas a large spread means most of the changes in individual hazard can be attributed to the exponential term (which is what we want). We can also view the coefficients and their ranges. [7]

```
from lifelines import CoxPHFitter
model = CoxPHFitter()
data['duration_col'] = duration_train
data['event_col'] = event_train
model.fit(data, duration_col='duration_col', event_col='event_col')
print(model.score_)
# plot the coefficients
model.plot()
# plot the survival function
model.predict_survival_function(test_data).plot()
```

- **Aalen's additive model**

Aalen's additive model assumes that the relationship between the t and the covariates x is of the following form:

$$\lambda(t|x) = b_0(t) + b_1(t).x_1 + \cdots + b_d(t).x_d$$

During the estimation, a linear regression is computed at each step. Sometimes the regression can be unstable (due to high co-linearity or small sample sizes), so adding a penalizer term in the model controls the stability. [8]

```
from lifelines import AalenAdditiveFitter
model = AalenAdditiveFitter(coef_penalizer=0.1)
data['duration_col'] = duration_train
data['event_col'] = event_train
model.fit(data, duration_col='duration_col', event_col='event_col')
# plot the cumulative hazards
model.plot()
# prediction
model.predict_survival_function(test_data).plot()
```

1. 3. Definition of the problem

Machine learning applications are at the forefront of the research in cancer prognosis. Most of the techniques employ machine learning methods for modeling the progression of cancer and identify informative factors that are used afterwards in a classification scheme.

In cases where physicians were forced to predict the survival and the life expectancy of a patient after an operation solely based on his/her histological and clinical data and on their experience, machine learning provides the formalized frame for prognosis.

The project in itself aims at providing personalized treatment to patients with skin cancer. This implies the ability of predicting a patient's survival after the different treatments that are available - but beforehand. That is the goal of this semester project: being able to predict the likelihood of a patient's survival after his/her operation and his/her life expectancy, based on his/her data such as age, gender, medical history, and based on the stained medical images of the patient's tissues (which would provide us with useful features such as the average distance between the cells, the mean size of the nuclei, etc.). Moreover, we would like to be able to cluster the population into groups based on these key factors.

However, because of some issues with the hospital, we have not been able to get the stained images of the already treated patients. Thus, this semester project deals in fact only with numerical data on the population and not images.

2. Visualization of the datasets

During this semester project, I dealt with two datasets. The first one was an Excel table provided by the hospital and the second one came from the website of a governmental health association.

These two datasets were quite similar, although their study population and the information they collected are not the same at all. I simply had to adapt the codes I wrote for the first dataset to the second one.

2.1. Pre-processing

The two datasets I have been working on had to be pre-processed. Their pre-processing differs not very much from each other.

- **First dataset**

I was given an Excel table filled with data on 74 patients. There were 62 columns, including useless columns such as the ID of the patients or whether they gave their consent to the study. Moreover, some cases of the table were empty and I had to see case by case whether the void was meaningful or not.

BRCA_status	HLA-class I	CD3 (high vs low)	CD3 (percentage)	CD8 (high vs low)	CD8 (percentage)	TLS
	1	0	1,37	0	0,33	1
	1	0	0,09	0	0,07	0
0	0	0	0,47	0	0,24	0
	1	0	0,6	0	0,82	0
0	2	0	0,67	0	0,19	0
0	1	0	0,09	0	0,07	0
	0	0	0,1	0	0,14	0
	1	1	5,22	1	3,61	0
	1	0	0,36	0	0,38	0
2	1	1	1,65	1	0,97	1
0	1	1	3,33	1	4	0
1	2	1	4	1	1,78	2
	1	1	repeat stain	1	1,44	
	0	1	4,18	1	1,48	2
1	2	1	1,1	1	0,67	

Fig.1: a glimpse of what the original Excel table given by the hospital looks like

In some columns, the empty cases were clearly due to a lack of information; but in others, after some research, it seemed logical that the absence of notation was voluntary, as it was for the column “BRCA_status”. It describes the status of the BRCA genes of each patient, as the presence of some mutations of either of the BRCA1 and

BRCA2 genes (which are tumor suppressor genes) is a significant factor for cancer prognosis. Here, “0” indicates that there are mutations on both genes BRCA1 and BRCA2, “1” indicates mutations only on BRCA1 and “2” indicates mutations only on BRCA2. It can be therefore easily deduced that the empty cases refer to patients that do not have any mutation on their BRCA genes.

Apart from these special columns, the empty cases of the other columns have been filled with the mean of the column so that I can use them without tampering much with the real data.

Multi-categorical columns had also to be taken care of. For the machine learning algorithms, it makes more sense to only have numerical and/or binary features (yes/no) than having multi-categorical features.

There are two essential columns for the problem: the “duration” and the “event” columns. The first one tells us how long they have been studying the patient if he/she is still alive or how long he/she survived since he/she entered the study if he/she is already dead. The second one gives the vital status of the patient at the last follow-up (1 if dead, 0 if alive).

➔ *See Code 1 (p.21) for the Python code*

- **Second dataset**

The second dataset has a much larger study population: it gathers data on 470 patients and has 61 columns. The meaning of each column was sometimes not so obvious: that is why I made some research on what each column means, based on its CDE identification. [9]

icd_10	icd_o_3_histolog	icd_o_3_site	informed_consent_verifi	patient_id	project_code	stage_other	tissue_source_site
icd_10	icd_o_3_histology	icd_o_3_site	int	patient_id	project_code	stage_other	tissue_source_site
CDE_ID:3226287	CDE_ID:3226275	CDE_ID:3226281	CI	CDE_ID:	CDE_ID:	CDE_ID:2007104	CDE_ID:
C44.5	8720/3	C44.5	YE	A9WB	[Not Available]	[Not Available]	3N
C77.3	8720/3	C77.3	YE	A9WC	[Not Available]	[Not Available]	3N
C77.0	8720/3	C77.0	YE	A9WD	[Not Available]	[Not Available]	3N
C44.9	8720/3	C44.9	YE	A1PU	[Not Available]	[Not Available]	BF
C44.9	8720/3	C44.9	YE	A1PV	[Not Available]	[Not Available]	BF
C44.9	8720/3	C44.9	YE	A1PX	[Not Available]	[Not Available]	BF
C44.9	8720/3	C44.9	YE	A1PZ	[Not Available]	[Not Available]	BF
C44.9	8720/3	C44.9	YE	A1Q0	[Not Available]	[Not Available]	BF
C44.9	8720/3	C44.9	YES	A3DJ	[Not Available]	[Not Available]	BF
C44.9	8720/3	C44.9	YES	A3DL	[Not Available]	[Not Available]	BF
C44.9	8720/3	C44.9	YES	A3DM	[Not Available]	[Not Available]	BF
C44.9	8720/3	C44.9	YES	A3DN	[Not Available]	[Not Available]	BF
C44.5	8720/3	C44.5	YES	A5EO	[Not Available]	[Not Available]	BF

Quartier et Cie:
The third edition of the International Classification of Diseases for Oncology, published in 2000, used principally in tumor and cancer registries for coding the site (topography) and the histology (morphology) of neoplasms._The description of an anatomical region or of a body part._Named locations of, or within, the body._A system of numbered categories for representation of data.

Fig.2: a portion of the second dataset and an example of the comments I added for more clarity

Apart from the fact that this preliminary research gave me a better understanding of the dataset, it also allowed me to discard some columns that are useless for us. Of course, columns like “form_completion_date” or “project_code” are obviously irrelevant to our project. But without this prior research, I would have kept other useless columns: for example, “tissue_source_site”, which simply refers to the TSS (Tissue Source Site) to which the samples of each patient have been sent to.

Furthermore, patients that have missing or incorrect information for their vital status and/or life duration had to be deleted, as these two pieces of information are essential. For example, there were patients that were indicated as still alive, but with negative life duration (after research, it seems that negative lifetimes are not some kind of code for the doctors): these patients had to be discarded from the dataset.

The rest of the pre-processing part is the same as for the first dataset.

➔ *Code 2 (p.21) for the Python code*

2. 2. A simple logistic regression: prediction of the survival

My first attempt was to do a logistic regression so that I can predict the probability that a patient survives after an operation. I split the dataset into a training set and a test set with the standard proportions: 80%-20%. Then, I trained the logistic model on the training set, then computed the scores of the model on both the training set and the test set.

For the first dataset, the training score was 100%, but the test score was 40%. Because of the small size of the dataset, the model was very likely to overfit - and that is what happened. It achieved a perfect score on the training set but failed to capture the “general” pattern of the dataset and was even worse than a random predictor (which would have a score around 50%).

For the second dataset, the training score was 87.8% and the test score was 79.8%, which is quite good.

➔ *Code 3 (p.23) for the Python code*

However, it appears that a simple logistic regression is not enough for the problem. Indeed, the knowledge of the vital status of the patients (dead or alive) is not subordinated to a fixed duration as all the patients did not join the study at the same time. In both datasets, there are people who have been studied for a short time and who are still alive and there are people who have been studied for a long time before dying. A logistic regression would have been relevant if the vital status of the patients was given at a fixed time T, so that we could predict for a future patient his/her probability to be alive at time T. That is why we have to use other tools to predict the survival of a patient after an operation.

2. 3. Comparison between Kaplan-Meier and Weibull models

I wanted to visualize the data and compare the two models of survival analysis: the Kaplan-Meier and Weibull models. Both estimates the survival function of the study

population, so at time t, they both compute the probability that the average patient is still alive; but the first one is non-parametric and the second one fits an exponential function to the data. Here are the two estimators computed for the two datasets on Fig.3.

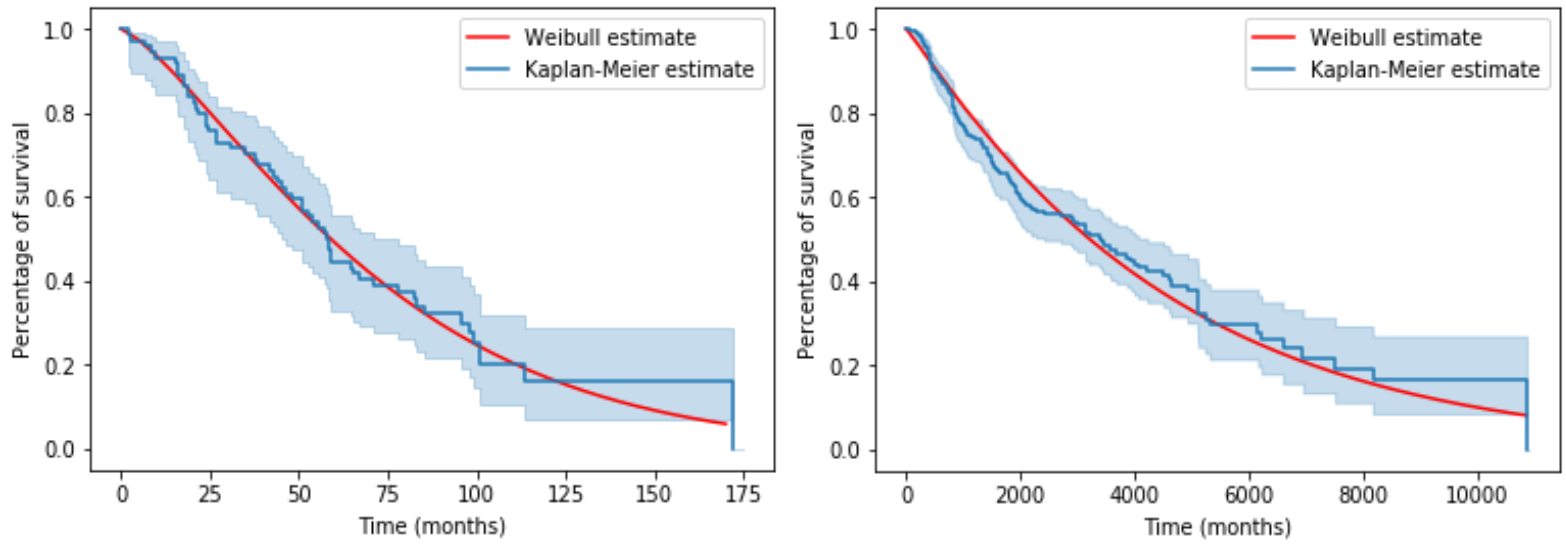


Fig.3: Comparison between the Kaplan-Meier estimator and the Weibull estimator (first and second datasets). The shaded area represents the confidence interval of the Kaplan-Meier estimate.

A Weibull analysis of the study data is compared to the Kaplan-Meier estimate of patient survival in both figures. It is clear from the Fig.3 that the Weibull model smooths the Kaplan-Meier estimator a lot but fits quite well (except at the end: that is explained by the fact that at the end, there are very few people who have been studied so long. That is why the Kaplan-Meier estimate is quite “rectangular” at the end). However, it would be more useful to consider more flexible models that take into consideration possible changes in hazard over various periods after initiation of therapy. In addition, detection of times where the risk of death changes sharply (changepoints) has broad implications for the management of the patients.

➔ *See Code 4 (p.24) for the Python code*

2. 4. Selection of the model

The Cox Proportional Hazard model requires a strong hypothesis: the proportional hazards hypothesis, which states that covariates are multiplicatively related to the hazard.

A quick and visual way to check the proportional hazards assumption of a variable is to plot the survival curves segmented by the values of a variable. If the survival curves are the same “shape” and differ only by a constant factor, then the assumption holds. A clearer way to see this is to plot the logs curves. If the curves are parallel (and hence do not cross each other), then it’s likely the variable satisfies the assumption.

Unfortunately, our dataset does not satisfy the proportional hazards assumption, as shown in Fig.4. Thus, I will only use the Aalen Additive model with different penalizer coefficients.

➔ *See Code 5 (p.25) for the Python code*

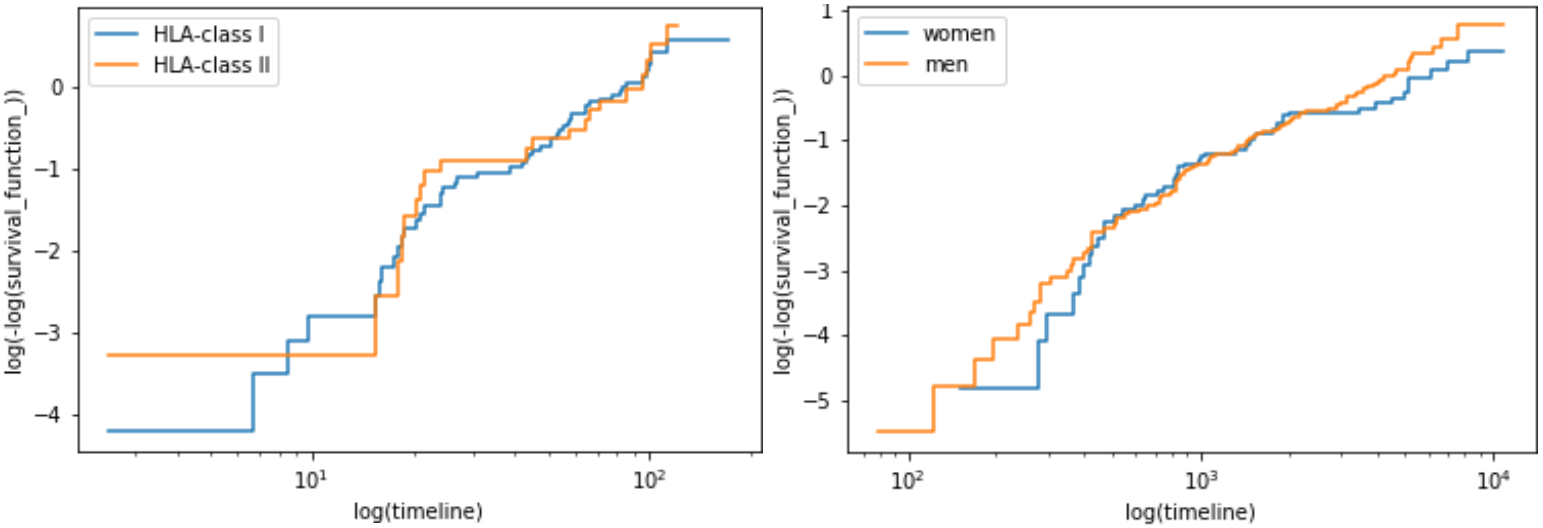


Fig.4: loglogs curves of two values of a same variable (first and second datasets)

3. Results for the first dataset

3.1. First method: dimensionality reduction

As we saw, the first dataset has more features (104 after pre-processing because of the multi-categorical columns) than patients (74). It is therefore natural to try to reduce the number of features so that then we only have to deal with a small number of meaningful features.

I applied a PCA (Principal Component Analysis) on the dataset. The PCA is an algorithm which uses an orthogonal transformation to convert our set of observations of variables that are certainly correlated into a set of values of linearly uncorrelated variables, called principal components. In practice, this algorithm consists in computing the eigenvectors and the eigenvalues of the covariance matrix of our set of observations, sorting the eigenvectors by decreasing eigenvalues, transposing our dataset into the new base, then keeping only the meaningful columns, i.e. by discarding the columns that do not vary much from a patient to another.

After PCA, the problem which had 104 columns after pre-processing was reduced to a 7-dimension problem.

➔ *See Code 6 (p.26) for the Python code*

Then, I clustered the data to show if there are some conditions that affects deeply the survival of a patient. To do that, I applied the K-Means algorithm to the data (without the columns of vital status and duration) and found the optimal number k of clusters with the elbow method. Here, the optimal k was 2, but even for k=2 the clustering is not what we are looking for:

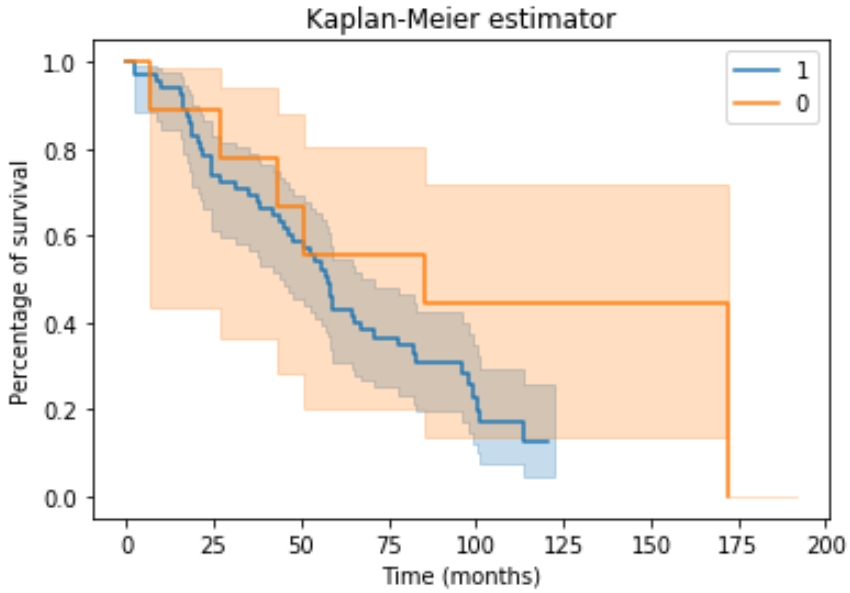


Fig.5: Kaplan-Meier estimator for the two clusters found by K-Means

What we wanted was to cluster the data into n populations so that we have n very “different” Kaplan-Meier estimators, or more scientifically, n completely separate Kaplan-Meier estimators of which the confidence intervals do not overlap on each

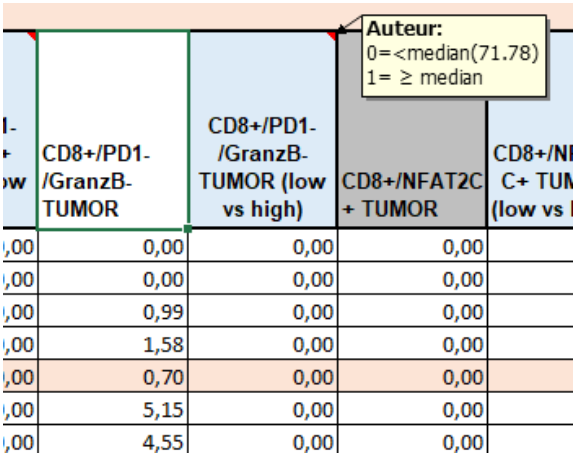
other. Thus, we could have said something like “if the patient has a very low concentration of plasma cells then he/she is likely not to survive very long after this operation, we should rather try another operation” etc. Here, the two shaded areas are superimposed over each other, and we can notice that the Kaplan-Meier estimate for the cluster “0” is very “rectangular” compared to the other one: that is due to the fact that there are not many patients in cluster “0” (only 9 to be precise, whereas there are 65 patients in cluster “1”).

➔ *See Code 7 (p.27) for the Python code*

Furthermore, because of the PCA, we lost the interpretability of the data which was a requirement of the hospital.

3.2. Second method: pre-selection of the data

In this dataset, there are in fact many columns that are merely deductions of others. For example, for each column with percentages (of cells, or tumors, etc.), there is sometimes a second column with 1s and 0s depending on whether the percentage is low or high. This second column has been created by comparing the values of the first column with the mean of the first column: it does not add any information. Therefore, I selected the columns that actually add information on the patients by discarding the columns that are mere duplicates of others. I did the change in the pre-processing part.



	CD8+/PD1-/GranzB-TUMOR	CD8+/PD1-/GranzB-TUMOR (low vs high)	CD8+/NFAT2C+ TUMOR	CD8+/NF C+ TUM (low vs
,00	0,00	0,00	0,00	
,00	0,00	0,00	0,00	
,00	0,99	0,00	0,00	
,00	1,58	0,00	0,00	
,00	0,70	0,00	0,00	
,00	5,15	0,00	0,00	
,00	4,55	0,00	0,00	

Fig.6: example of duplicate columns

➔ *See Code 8 (p.28) for the Python code*

Because we don’t want to lose the interpretability of the data, I could not apply the PCA algorithm. So, as previously, I tried to cluster the data into meaningful groups:

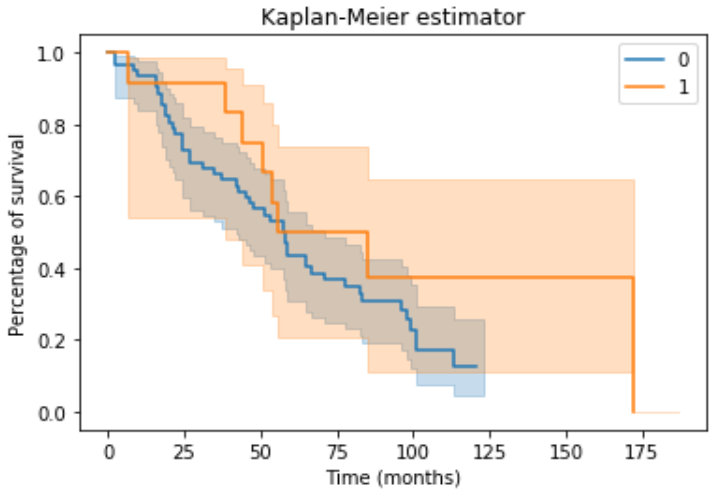


Fig.7: Kaplan-Meier estimator for the two clusters found by K-Means (pre-selection)

In fact, it doesn’t change really anything. The process was just a little longer because of the higher dimensionality of the data. As previously, the clustering didn’t

yield the results we wanted on the data: the two graphs are not well differentiated and the confidence intervals overlap. That is why we have to take into account the information on the patients directly in the estimation of their survival function by using tools from the survival regression (Aalen's additive model), and not only from the survival analysis (Kaplan-Meier model) anymore.

➔ *See Code 9 (p.30) for the Python code*

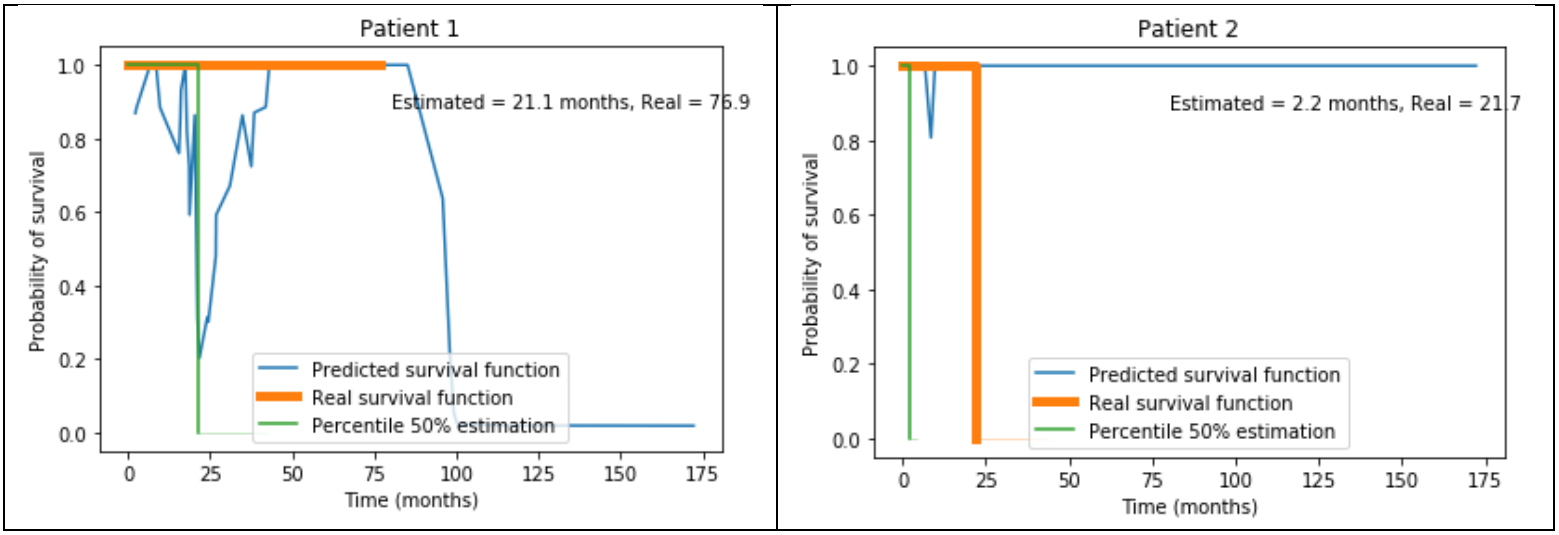
3.3. Third method: survival regression

The aim is now to predict directly the survival function of the patients after training the Aalen's additive model. We don't want to apply PCA so that we keep the interpretability of the data for the hospital. So, I will train the model on the data after discarding the columns that are duplicates of others (part 3.2.).

The problem is that we have now too many columns (75) compared to the number of patients (74). Splitting the already small dataset into a training set and a test set would lead to overfitting. That is why I used the LOOCV (Leave-One-Out Cross-Validation) to visualize the results: that is a specific situation of the k-fold cross validation, where the original sample is divided into k samples, then we select each one of the k samples as the test set and the k-1 others as the training set. Here, for each patient, I will train the model on all the other patients and compute the survival function predicted by the model for this patient (k is equal to the number of patients).

➔ *See Code 10 (p.31) for the Python code*

As we can see below on Fig.8, the survival functions predicted are very wiggly. We also computed the median of the survival functions, namely the (first) time at which the patient has 50% chance of dying.



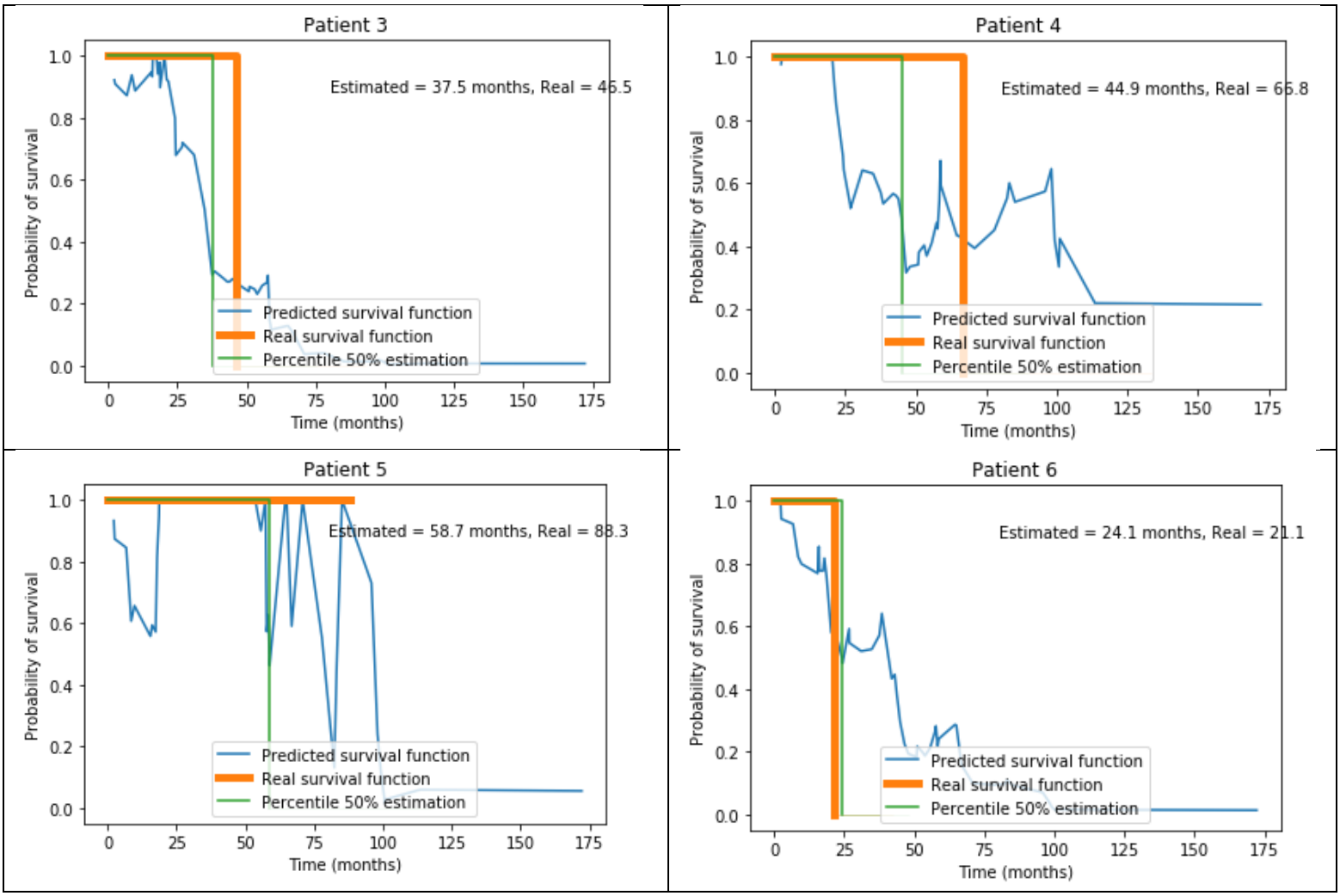


Fig.8: predicted survival functions for the first 6 patients (LOOCV, Aalen's additive model)

The survival functions should be a decreasing function of time, which is not the case here. The predictions are not very good, even if we only take the median of the survival function as the estimated life expectancy: if the patient is still alive, the estimated duration should be bigger than the "real" duration (for which the patient has been studied), and if the patient is already dead, the estimated duration should be the same as the "real" duration. It is rarely the case if we visualize them for the whole dataset.

To deal with the issue of the survival functions not being decreasing functions, I tried to fit a logistic function to the predicted survival functions above. The logistic function has the following equation:

$$y = \frac{1}{1 + \exp(-k(x - x_0))}$$

where k and x_0 are the parameters of the logistic function. The logistic function that fits the best to the survival function is the one that minimizes the least square error.

➔ *See Code 11 (p.32) for the Python code*

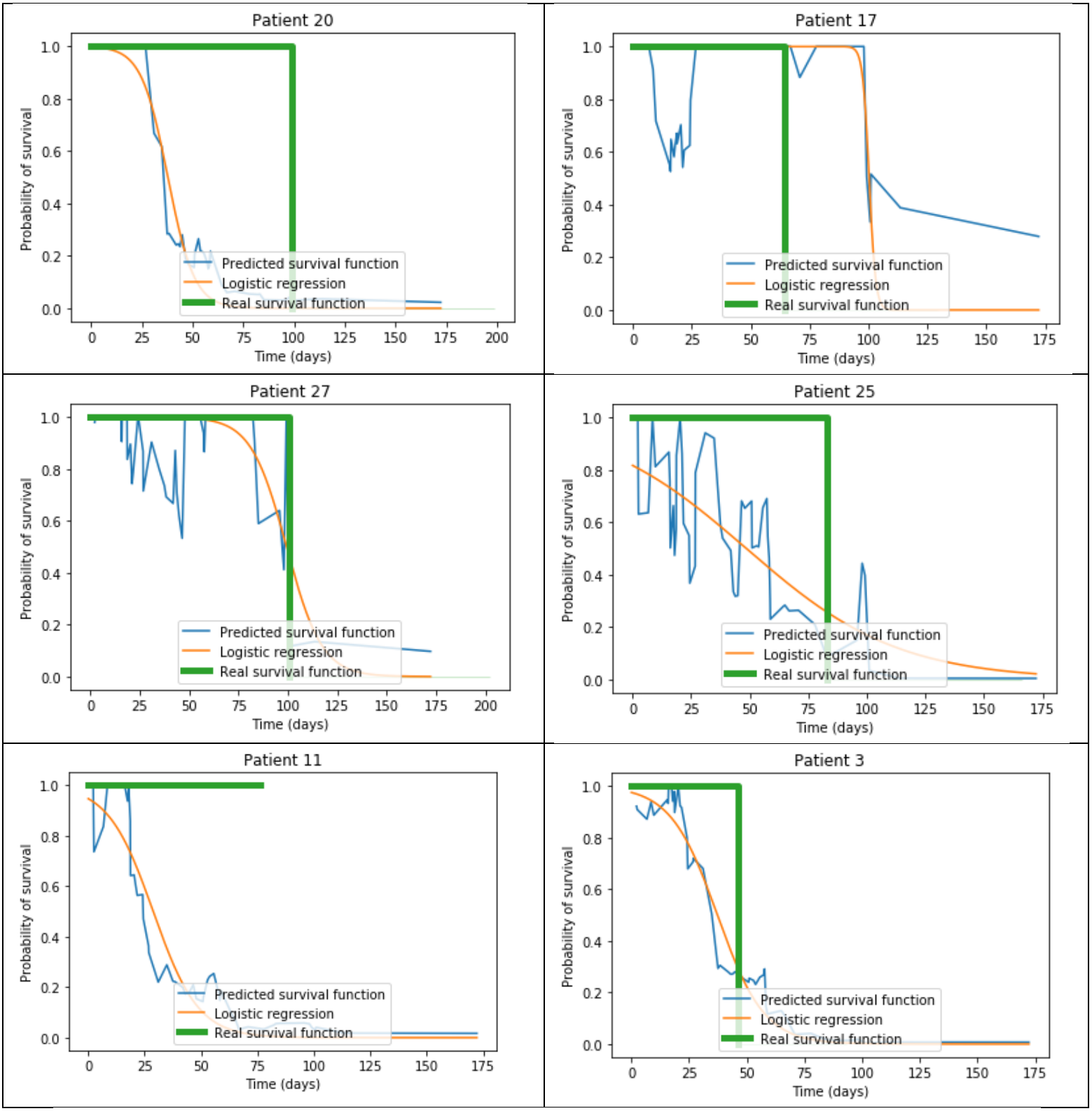


Fig.9: predicted survival functions (smoothed) for 6 randomly chosen patients (LOOCV, Aalen's additive model)

Although the smoothed-out survival functions succeeded in capturing the general form of the predicted survival functions, in fact, we can notice that it doesn't really match with the real survival function.

4. Results for the second dataset

4.1. First method: simple clustering

After pre-processing the second dataset, we applied the K-Means algorithm to the features (without the two columns “duration” and “event”) so that we can cluster the data into two (or more) groups with separate Kaplan-Meier estimates.

➔ *See Code 12 (p.34) for the Python code*

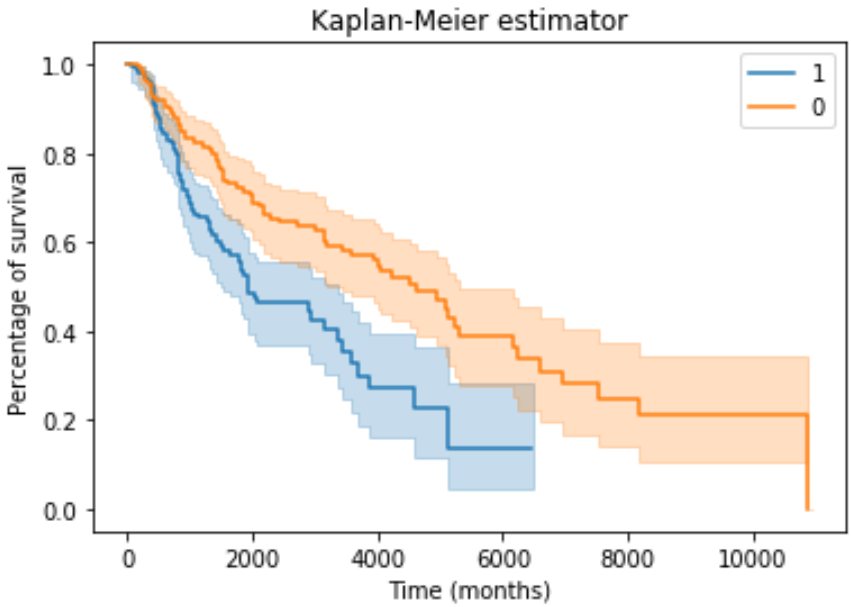


Fig.10: Kaplan-Meier estimators for the two clusters found by K-Means (second dataset)

As we can see on Fig.10, the results are quite good. The Kaplan-Meier estimates of the two clusters are quite well differentiated and the two confidence intervals don’t overlap too much on one another. Moreover, there are 227 patients in cluster “1” and 214 patients in cluster “0”, so it is balanced in contrast to the first dataset.

4.2. Second method: survival regression

I fitted the Aalen’s additive model to the second dataset, so that we can directly predict the survival functions of the patients. As for the first dataset, I used the LOOCV to compute the survival functions of the first six patients.

➔ *See Code 13 (p.35) for the Python*

On Fig.11, the survival functions of the first six patients are displayed. The “estimated” life expectancy after the operation is the first time at which the probability of dying is of 50%, the “real” life expectancy indicates how long the person has been

studied if he/she is still alive, and how long it survived after entering the study if she/he is already dead.

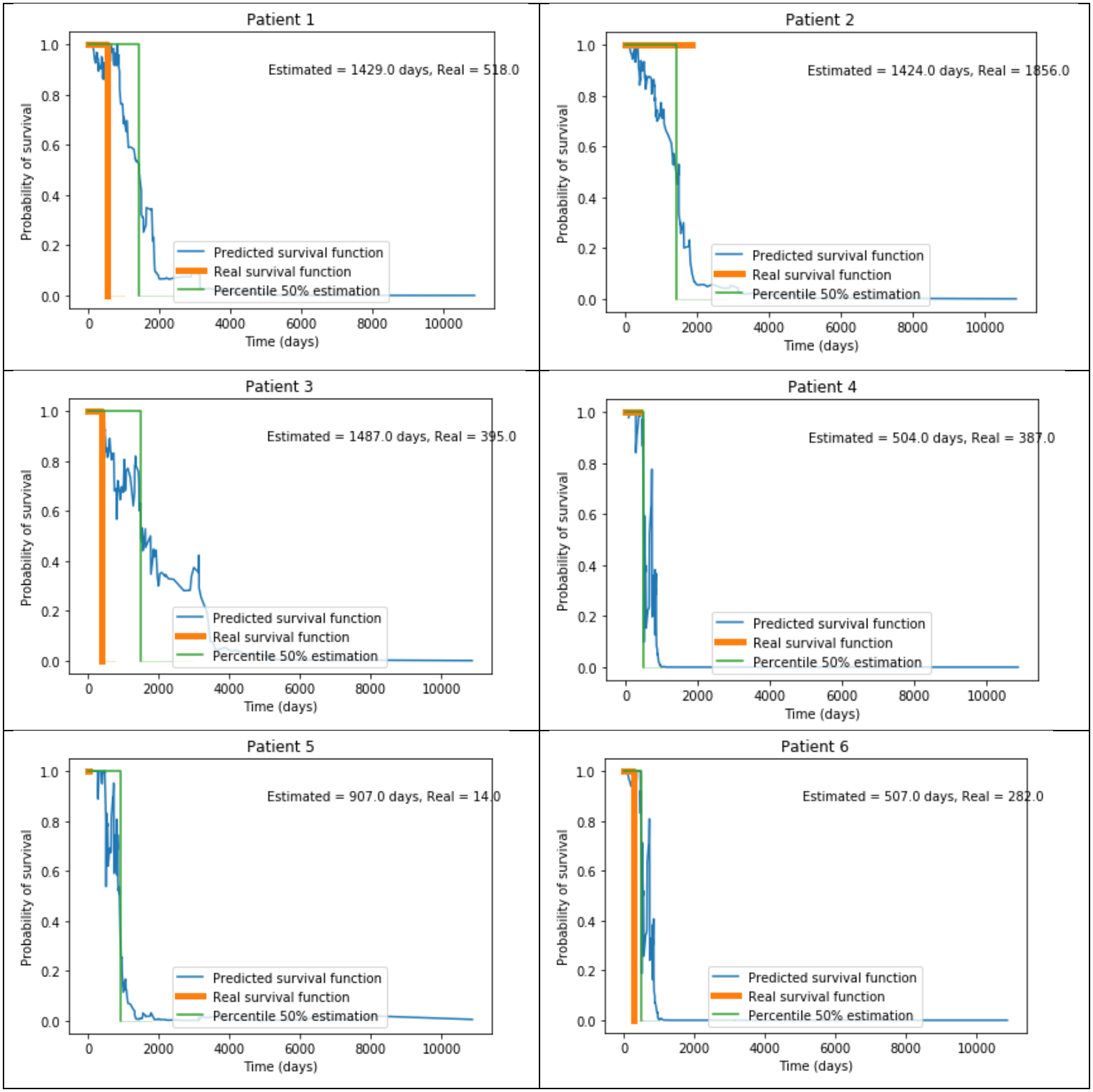


Fig.11: predicted survival functions for the first six patients (second dataset, LOOCV)

It rises the same problem as for the first dataset, namely the fact the predicted survival functions are “wiggly” and are not decreasing functions. So, like for the first dataset, I tried to fit a logistic function to theses survival functions. We can notice that

as there are six times more patients in the second dataset than in the first dataset, although they are still quite fluctuating, they seem more stable.

➔ *See Code 14 (p.37) for the Python*

The smoothed out predicted survival functions for the first six patients are displayed on Fig.12.

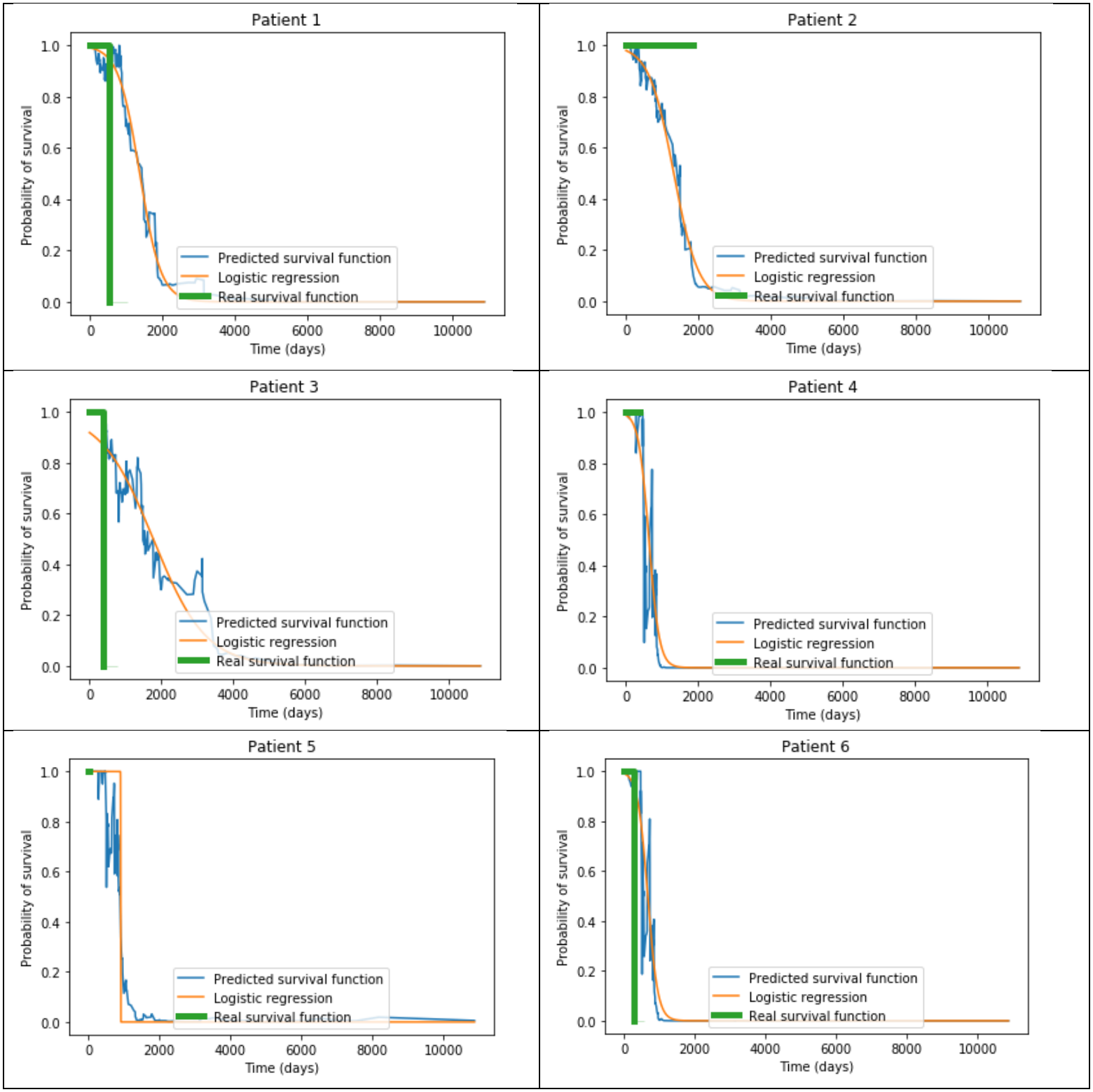


Fig.12: smoothed predicted survival functions for the first six patients (second dataset, LOOCV)

It remains quite random. I visualized the smoothed predicted survival functions for the whole dataset using LOOCV but it did not really have a great correlation with the real survival functions. I had different criteria depending on the vital status of the patients:

- for patients who are already dead, I looked if the estimated life expectancy matches with the real life duration
- for patients who are still alive, I looked if the estimated life expectancy is larger than the length of time during which the patients have been studied.

It did not yield good results. For patients that are still alive, it achieved a score of 36.7%, which is worse than the score of a random predictor. For patients that are already dead, the average error is 119%.

5. Conclusion

In this semester project, I used tools provided by the survival analysis in order to realize prognosis on patients with skin cancer. This is supposed to help the doctors choose the most suitable treatment for each patient by comparing the different life expectancies obtained for each available treatment. The best way to do so was to cluster the data into groups of very different life expectancies and small confidence intervals so that we can predict approximately the life expectancy of a patient with his/her information. Indeed, trying to predict directly the survival function of a patient with survival regression has proven itself not very relevant as keeping them as wiggly as they are would not be correct and smoothing them out with a logistic function makes them lose their information.

6. Appendix

6.1. Python codes

- **Code 1: pre-processing (first dataset)**

```
import pandas

__all__ = ['data', 'duration_col', 'event_col']
table = pandas.read_excel('tableau.xlsx', skiprows=[0])

multicategorical_columns = ['BRCA_status.1','HLA-class I', 'TLS', 'plasma cells','
plasma cells (IHC=CD138)']
ignore = ['Block No', 'Block No.1', 'BRCA_status']

# if there is an empty square on BRCA_status that means there is no mutation
table['BRCA_status.1'].fillna(3.0,inplace=True)

duration_col = 'New analysis by JANOS Survival (months)'
event_col = 'New analysis by JANOS dead (1)/alive (0)'

data = pandas.DataFrame()

for c in table.columns:
    if c in ignore:
        continue
    elif c in multicategorical_columns:
        x = pandas.get_dummies(table[c], prefix=c)
        data[x.columns] = x
    else:
        # coerce to convert strings to NaN
        data[c] = pandas.to_numeric(table[c], errors='coerce')
        # replace NaN by the average value of the column
        data[c].fillna(data[c].mean(), inplace=True)
```

- **Code 2: pre-processing (second dataset)**

```
import pandas
```

```
import numpy as np

__all__ = ['data', 'duration_col', 'event_col']
table = pandas.read_excel('tableau_nouveau.xlsx', skiprows=[0,1,3])

numerical_columns = ['days_to_birth', 'height', 'weight',
'primary_melanoma_at_diagnosis_count', 'breslow_depth_value',
'malignant_neoplasm_mitotic_count_rate', 'age_at_initial_pathologic_diagnosis',
'days_to_submitted_specimen_dx', 'primary_anatomic_site_count']
ignore =
['bcr_patient_uuid','bcr_patient_barcode','form_completion_date','year_of_initial_
pathologic_diagnosis','days_to_initial_pathologic_diagnosis','informed_consent_ver
ified','patient_id','tissue_source_site', 'system_version']

data = pandas.DataFrame()

# total number of patients
nb_patients = table.shape[0]

# extraction of event_col (dead 1/alive 0)
event_col = 'event_col'
data[event_col] = table['vital_status'].replace('Dead', 1).replace('Alive',0)
data[event_col] = pandas.to_numeric(data[event_col], errors='coerce')
data[event_col].fillna(int(data[event_col].mean()), inplace=True)

# extraction of duration_col
duration_col = 'duration_col'
skiprow = list()
data[duration_col] = pandas.to_numeric(table['days_to_death'], errors='coerce')
y = pandas.to_numeric(table['days_to_last_followup'], errors='coerce')
for i in range(nb_patients):
    alive = y[i]
    dead = data[duration_col][i]
    if np.isnan(alive)^np.isnan(dead):
        data[duration_col][i] = np.nan_to_num(alive) + np.nan_to_num(dead)
        if alive<=0 or dead<=0:
            skiprow.append(i)
    else:
        skiprow.append(i)
data.drop(skiprow, inplace=True)
```

```
print()
print("List of the patients of which the survival time is missing or incorrect:
\n", skiprow)
print()
print('-'*60)


# don't need the three columns any more in table
ignore += ['vital_status', 'days_to_last_followup', 'days_to_death']


for c in table.columns:
    if c in ignore:
        continue
    elif c in numerical_columns:
        # coerce to convert strings to NaN
        data[c] = pandas.to_numeric(table[c], errors='coerce')
        # replace NaN by the mean of the column
        data[c].fillna(data[c].mean(), inplace=True)
    else:
        x = pandas.get_dummies(table[c], prefix=c)
        data[x.columns] = x


# drop all the columns [Not Available], [Not Evaluated], etc...
data.drop(columns = [col for col in data if '[' in col], inplace=True)
data = data.reset_index(drop=True)
```

- **Code 3: Prediction of the survival after an operation**

```
from data import data, event_col # repeat for both datasets
from sklearn import linear_model
import numpy as np

# Is the patient dead or alive after an operation?
def dead_or_alive(x):
    if x:
        return "dead"
    return "alive"
```

```
def survive(training_set, test_set):
    X_train = np.array(training_set.iloc[:, 2:])
    y_train = np.array(training_set[event_col])
    clf = linear_model.LogisticRegression(C=1e5, fit_intercept = False)
    clf.fit(X_train, y_train)
    X_test = np.array(test_set.iloc[:, 2:])
    y_test = np.array(test_set[event_col])
    train_score = clf.score(X_train, y_train)
    test_score = clf.score(X_test, y_test)
    return train_score, test_score

# split training/test set : 80%/20%
s = int(0.8*data.shape[0])
training_set = data.iloc[:s]
test_set = data.iloc[s:]
train_score, test_score = survive(training_set, test_set)

print("Score on the training set: {:3.1f} %".format(train_score*100))
print("Score on the test set: {:3.1f} %".format(test_score*100))
```

- **Code 4: comparison between the Kaplan-Meier and the Weibull models**

```
import matplotlib.pyplot as plt
import numpy as np
from lifelines import KaplanMeierFitter, WeibullFitter

from data import *  # repeat with both datasets

kmf = KaplanMeierFitter()
wf = WeibullFitter()

# Weibull model
wf.fit(data[duration_col], data[event_col])
print('Weibull model:')
print()
print('Lambda: ',wf.lambda_,'Rho: ', wf.rho_) # Weibull parameters
t = np.arange(0.0, np.max(data[duration_col]), 5.0)
ax = plt.subplot(111)
```

```
ax.plot(t, np.exp(-(wf.lambda_*t)**wf.rho_), 'r', label = 'Weibull model')

# Kaplan-Meier model
kmf.fit(data[duration_col],data[event_col])
kmf.plot(ax=ax)
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles, ['Weibull estimate', 'Kaplan-Meier estimate'])
plt.xlabel('Time (months)')
plt.ylabel('Percentage of survival')
```

- **Code 5: proof that the proportional hazards hypothesis is false**

```
import matplotlib.pyplot as plt
from lifelines import KaplanMeierFitter

# First dataset
from data1 import *

HLA_0 = data.loc[data['HLA-class I_0.0'] == 0.0]
HLA_1 = data.loc[data['HLA-class I_1.0'] == 1.0]

kmf0 = KaplanMeierFitter()
kmf0.fit(HLA_0[duration_col], event_observed=HLA_0[event_col])

kmf1 = KaplanMeierFitter()
kmf1.fit(HLA_1[duration_col], event_observed=HLA_1[event_col])

fig, axes = plt.subplots()
kmf0.plot_loglogs(ax=axes)
kmf1.plot_loglogs(ax=axes)

axes.legend(['HLA-class I', 'HLA-class II'])
plt.show()

# Second dataset
from data10 import *

woman = data.loc[data['gender_FEMALE'] == 1.0]
man = data.loc[data['gender_MALE'] == 1.0]
```

```
kmf0 = KaplanMeierFitter()
kmf0.fit(woman[duration_col], event_observed=woman[event_col])

kmf1 = KaplanMeierFitter()
kmf1.fit(man[duration_col], event_observed=man[event_col])

fig, axes = plt.subplots()
kmf0.plot_loglogs(ax=axes)
kmf1.plot_loglogs(ax=axes)

axes.legend(['women', 'men'])
plt.show()
```

- **Code 6: PCA for the first dataset**

```
import numpy as np
import numpy.linalg
import pandas as pd

from data1 import *

__all__ = ['new_data']

def PCA(mat):
    # computes the covariance matrix
    mat = mat - mat.mean(axis=0)
    cov = mat.T.dot(mat)
    # computes eigenvectors and corresponding eigenvalues
    val, vect = numpy.linalg.eigh(cov)
    # sorts the eigenvectors by decreasing eigenvalues
    x = val.argsort()[::-1]
    val, vect = val[x], vect[:, x]
    # verification
    eps = ((vect * val).dot(vect.T) - cov).mean()
    assert eps < 1e-9, eps
    # new coordinates
    ncoords = mat.dot(vect)
    return ncoords, vect, np.cumsum(val), val
```

```
# dimension reduction with PCA
X = data.drop([duration_col, event_col], axis=1)
nX, _, cum, _ = PCA(X.as_matrix())
# determines the number of eigenvalues the sum of which is ~the final sum
dim = np.argmax(cum / cum[-1] > .99)
nX = nX[:, :dim]
# new data
new_data = pd.DataFrame(nX)

for i in range(10):
    print()
print('-' * 60)
print()
print("Reduced dimension to", dim)
print()
```

- **Code 7: clustering of the data after PCA**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from scipy.spatial.distance import cdist
from lifelines import KaplanMeierFitter
import pandas as pd

from data1 import duration_col, event_col
from script1 import new_data # data after PCA

# Clustering the data
distortions = list()
K_values = range(1,10)
for k in K_values:
    kmeanModel = KMeans(n_clusters=k).fit(new_data)
    dis = sum(np.min(cdist(new_data, kmeanModel.cluster_centers_,'euclidean'),axis
=1))
    distortions.append(dis/new_data.shape[0])
plt.plot(K_values,distortions,'bx-')
```

```
plt.xlabel('k')
plt.ylabel('distortion')
plt.title('Elbow method showing optimal k')
plt.show()

# Optimal number of clusters?
k_opt=int(input("Entrez la valeur optimale pour le nombre de clusters k : "))

# Final clustering
new_data[duration_col] = data[duration_col]
new_data[event_col] = data[event_col]
kmeanModel = KMeans(n_clusters=k_opt).fit(new_data)
labels = pd.DataFrame(kmeanModel.labels_)
new_data["labels"] = labels
print(kmeanModel.cluster_centers_)

# Kaplan-Meier estimator
kmf = KaplanMeierFitter()
groups = new_data.iloc[:,-1]
ax = plt.subplot(111)
for group in groups.unique():
    ix = (groups==group)
    kmf.fit(new_data[duration_col][ix],new_data[event_col][ix],label=group)
    ax = kmf.plot(ax=ax)
plt.ylabel('Percentage of survival')
plt.xlabel('Time (months)')
plt.title('Kaplan-Meier estimator')
plt.show()
```

- **Code 8: pre-selection by hand of the relevant columns**

```
import pandas
__all__ = ['data', 'duration_col', 'event_col']

table = pandas.read_excel('tableau.xlsx', skiprows=[0])

multicategorical_columns = ['BRCA_status.1','HLA-class I', 'TLS', 'plasma cells','
plasma cells (IHC=CD138)']
```

```
ignore = ['Block No', 'Block No.1', 'BRCA_status','CD3 (percentage)','CD8 (percentage)','CD8+ TUMOR','CD8+/PD1+ TUMOR','CD8+/GranzB+ TUMOR','CD8+/PD1+/GranzB+ TUMOR','CD8+/PD1+/GranzB- TUMOR','CD8+/PD1-/GranzB+ TUMOR','CD8+/PD1-/GranzB- TUMOR','CD8+/NFAT2C+ TUMOR','percentage of CD8+/PD-1+ in CD8+','percentage of CD8+/GranzB+ in CD8+','percentage of CD8+/PD-1+/GranzB+ in CD8+','percentage of CD8+/PD-1+/GranzB+ in CD8+/PD-1+','percentage of CD8+/PD-1+/GranzB- in CD8+/PD-1+','percentage of CD8+/PD-1+/GranzB+ in CD8+/GranzB+','percentage of CD8+/PD-1-/GranzB+ in CD8+/GranzB+','percentage of CD8+/PD-1+/GranzB- in CD8+','percentage of CD8+/PD-1-/GranzB+ in CD8+','percentage of CD8+/PD-1-/GranzB- in CD8+','percentage of CD8+/NFAT2c+ in CD8+','CD8+ STROMA','CD8+/PD1+ STROMA','CD8+/GranzB+ STROMA','CD8+/PD1+/GranzB+ STROMA','CD8+/PD1+/GranzB- STROMA','CD8+/PD1-/GranzB+ STROMA','CD8+/PD1-/GranzB- STROMA','CD8+/NFAT2C+ STROMA','percentage of CD8+/PD-1+ in CD8+','percentage of CD8+/GranzB+ in CD8+','percentage of CD8+/PD-1+/GranzB+ in CD8+','percentage of CD8+/PD-1+/GranzB+ in CD8+/PD-1+','percentage of CD8+/PD-1+/GranzB- in CD8+/PD-1+','percentage of CD8+/PD-1+/GranzB+ in CD8+/GranzB+','percentage of CD8+/PD-1-/GranzB+ in CD8+/GranzB+','percentage of CD8+/PD-1+/GranzB- in CD8+','percentage of CD8+/PD-1-/GranzB+ in CD8+','percentage of CD8+/PD-1-/GranzB- in CD8+','percentage of CD8+/NFAT2c+ in CD8+']

# if there is an empty square on BRCA_status that means there is no mutation
table['BRCA_status.1'].fillna(3.0,inplace=True)

duration_col = 'New analysis by JANOS Survival (months)'
event_col = 'New analysis by JANOS dead (1)/alive (0)'

data = pandas.DataFrame()

for c in table.columns:
    if c in ignore:
        continue
    elif c in multicategorical_columns:
        x = pandas.get_dummies(table[c], prefix=c)
        data[x.columns] = x
    else:
        # coerce to convert strings to NaN
        data[c] = pandas.to_numeric(table[c], errors='coerce')
        # replace NaN by the average value of the column
        data[c].fillna(data[c].mean(), inplace=True)
```

- **Code 9: clustering of the data after pre-selection**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from scipy.spatial.distance import cdist
from lifelines import WeibullFitter
from lifelines import KaplanMeierFitter
from lifelines import CoxPHFitter, AalenAdditiveFitter
from lifelines.utils import k_fold_cross_validation
import pandas as pd

from data2 import * # data after pre-selection
new_data = data

# Clustering the data
distortions = list()
K_values = range(1,10)
for k in K_values:
    kmeanModel = KMeans(n_clusters=k).fit(new_data)
    dis = sum(np.min(cdist(new_data, kmeanModel.cluster_centers_,'euclidean'),axis
=1))
    distortions.append(dis/new_data.shape[0])
plt.plot(K_values,distortions,'bx-')
plt.xlabel('k')
plt.ylabel('distortion')
plt.title('Elbow method showing optimal k')
plt.show()

# Optimal number of clusters?
k_opt=int(input("Entrez la valeur optimale pour le nombre de clusters k : "))

# Final clustering
kmeanModel = KMeans(n_clusters=k_opt).fit(new_data)
labels = pd.DataFrame(kmeanModel.labels_)
new_data = pd.concat([new_data,labels],axis=1)
print(kmeanModel.cluster_centers_)

# Kaplan-Meier estimator
```

```
kmf = KaplanMeierFitter()
groups = new_data.iloc[:,-1]
ax = plt.subplot(111)
for group in groups.unique():
    ix = (groups==group)
    kmf.fit(new_data[duration_col][ix],new_data[event_col][ix],label=group)
    ax = kmf.plot(ax=ax)
plt.ylabel('Percentage of survival')
plt.xlabel('Time (months)')
plt.title('Kaplan-Meier estimator')
plt.show()
```

- **Code 10: prediction of the survival function of the patients**

```
from data2 import*
from lifelines import AalenAdditiveFitter, KaplanMeierFitter
import matplotlib.pyplot as plt
import numpy as np



# models
aaf = AalenAdditiveFitter(coef_penalizer=.1)
kmf1 = KaplanMeierFitter()
kmf2 = KaplanMeierFitter()
models = [aaf]



# prediction for aaf
def prediction(model, data, i, drop=True):
    if drop:
        data_bis = data.drop([i])
    else:
        data_bis = data
    try:
        model.fit(data_bis, duration_col=duration_col, event_col=event_col,
show_progress=True)
    except Exception:
        print("There is a problem")
    axis = plt.subplot(111)
```

```
    axes.append(axis)
    coord = model.predict_survival_function(data.iloc[i:i+1,2:])
    coord.plot(ax=axis)
    coord = coord.reset_index().values
    predicted = coord[:,0][np.argmin(coord[:,1]>0.5)]
    real = data.iloc[i,1]
    axis.annotate("Estimated = {} months, Real = {}".format(predicted, real), xy =
(.47, .85), xycoords = "axes fraction")
    if str(predicted) == 'inf':
        event = 0.
        predicted = 200. # maximal duration of the study
    else:
        event = data[event_col][i]
    kmf1.fit(np.array([data[duration_col][i]]), np.array([data[event_col][i]]))
    kmf1.plot(ax=axis, linewidth=5.0)
    kmf2.fit(np.array([predicted]), np.array([1.]))
    kmf2.plot(ax=axis)
    plt.ylabel('Probability of survival')
    plt.xlabel('Time (months)')
    plt.title('Patient {}'.format(i+1))
    handles, _ = axis.get_legend_handles_labels()
    axis.legend(handles, ["Predicted survival function", "Real survival function",
"Percentile 50% estimation"], loc = 'lower center')
    plt.show()

for m in models:
    axes = list()
    for i in range(6): # only the 6 first patients
        prediction(m, data, i)
```

- **Code 11: smoothing out the predicted survival functions**

```
from data2 import data, event_col, duration_col
from lifelines import AalenAdditiveFitter, KaplanMeierFitter
import matplotlib.pyplot as plt
import scipy.optimize
import numpy as np
import random
```

```
# models
aaf = AalenAdditiveFitter(coef_penalizer=.1)
kmf = KaplanMeierFitter()
models = [aaf]

# logistic function
def sigmoid(p,x):
    x0, k = p
    y = 1 / (1 + np.exp(-k*(x-x0)))
    return y

# squared error
def residuals(p,x,y):
    return y - sigmoid(p,x)

# prediction
def prediction_smooth(model, data, i, drop=True, display=True):
    """Returns a decreasing survival function.
    Logisic function fitted to the predicted survival function returned by the
model."""
    if drop:
        data_bis = data.drop([i])
    else:
        data_bis = data
    try:
        model.fit(data_bis, duration_col=duration_col, event_col=event_col,
show_progress=True)
    except Exception:
        print("There is a problem")
    axis = plt.subplot(111)
    coord = model.predict_survival_function(data.iloc[i:i+1,2:])
    if display:
        coord.plot(ax=axis)
    coord = coord.reset_index().values
    predicted_i = np.argmin(coord[:,1]>max(0.05, np.min(coord[:,1])))
    x = coord[:predicted_i,0]
    y = coord[:predicted_i,1]
    p_guess = (coord[np.argmin(coord[:,1]>0.5), 0], -1.0)
    p = scipy.optimize.leastsq(func=residuals, x0=p_guess, args=(x,y),
full_output=1)[0]
```

```
    if display:
        xp = np.linspace(0, np.max(coord[:,0]), np.max(coord[:,0])*10)
        pxp = sigmoid(p,xp)
        axis.plot(xp, pxp, '-', label = "log")
        kmf.fit(np.array([data[duration_col][i]]), np.array([data[event_col][i]]))
        kmf.plot(ax=axis, linewidth=5.0)
        plt.ylabel('Probability of survival')
        plt.xlabel('Time (days)')
        plt.title('Patient {}'.format(i+1))
        handles, _ = axis.get_legend_handles_labels()
        axis.legend(handles, ["Predicted survival function", "Logistic
regression", "Real survival function"], loc = 'lower center')
        plt.show()
    return p[0] # median = life expectancy

for m in models:
    for i in range(20):
        k = random.randint(0, data.shape[0])
        prediction_smooth(m, data, k)
```

- **Code 12: clustering of the data (second dataset)**

```
import matplotlib.pyplot as plt
import numpy as np
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from scipy.spatial.distance import cdist
from lifelines import KaplanMeierFitter
import pandas as pd

from data10 import data, duration_col, event_col
new_data = data.drop(columns = [duration_col, event_col])

# Clustering the data
distortions = list()
K_values = range(1,10)
for k in K_values:
    kmeanModel = KMeans(n_clusters=k).fit(new_data)
```

```
    distortions.append(sum(np.min(cdist(new_data,kmeanModel.cluster_centers_,'eucl
idean'),axis=1))/new_data.shape[0])
plt.plot(K_values,distortions,'bx-')
plt.xlabel('k')
plt.ylabel('distortion')
plt.title('Elbow method showing optimal k')
plt.show()

# Optimal number of clusters?
k_opt = int(input("Please enter the optimal number of the clusters k: "))

# Final clustering
kmeanModel = KMeans(n_clusters=k_opt).fit(new_data)
labels = pd.DataFrame(kmeanModel.labels_)
new_data['labels'] = labels
print(kmeanModel.cluster_centers_)

# Kaplan-Meier estimator
new_data[event_col] = data[event_col]
new_data[duration_col] = data[duration_col]
kmf = KaplanMeierFitter()
groups = new_data['labels']
ax = plt.subplot(111)
for group in groups.unique():
    ix = (groups==group)
    kmf.fit(new_data[duration_col][ix],new_data[event_col][ix],label=group)
    ax = kmf.plot(ax=ax)
plt.ylabel('Percentage of survival')
plt.xlabel('Time (months)')
plt.title('Kaplan-Meier estimator')
plt.show()
```

- **Code 13: prediction of the survival function of the patients**

```
from data10 import data, event_col, duration_col
from lifelines import AalenAdditiveFitter, KaplanMeierFitter
import matplotlib.pyplot as plt
import numpy as np
```

```
# models
aaf = AalenAdditiveFitter(coef_penalizer=.1)
kmf1 = KaplanMeierFitter()
kmf2 = KaplanMeierFitter()
models = [aaf]

# prediction for aaf
def prediction(model, data, i, drop=True):
    if drop:
        data_bis = data.drop([i])
    else:
        data_bis = data
    try:
        model.fit(data_bis, duration_col=duration_col, event_col=event_col, show_p
rogress=True)
    except Exception:
        print("There is a problem")
    axis = plt.subplot(111)
    axes.append(axis)
    coord = model.predict_survival_function(data.iloc[i:i+1,2:])
    coord.plot(ax=axis)
    coord = coord.reset_index().values
    predicted = coord[:,0][np.argmin(coord[:,1]>0.5)]
    real = data.iloc[i,1]
    axis.annotate("Estimated = {} days, Real = {}".format(predicted, real), xy = (
.47, .85), xycoords = "axes fraction")
    if str(predicted) == 'inf':
        predicted = 10000.
    kmf1.fit(np.array([data[duration_col][i]]), np.array([data[event_col][i]]))
    kmf1.plot(ax=axis, linewidth=5.0)
    kmf2.fit(np.array([predicted]), np.array([1.]))
    kmf2.plot(ax=axis)
    plt.ylabel('Probability of survival')
    plt.xlabel('Time (days)')
    plt.title('Patient {}'.format(i+1))
    handles, _ = axis.get_legend_handles_labels()
    axis.legend(handles, ["Predicted survival function", "Real survival function",
"Percentile 50% estimation"], loc = 'lower center')
    plt.show()
```

```
for m in models:
    axes = list()
    for i in range(6):
        prediction(m, data, i)
```

- **Code 14: smoothing out the predicted survival functions**

```
import matplotlib.pyplot as plt
import numpy as np
from data10 import data, event_col, duration_col
from lifelines import AalenAdditiveFitter, KaplanMeierFitter
import matplotlib.pyplot as plt
import scipy.optimize
import numpy as np


# models
aaf = AalenAdditiveFitter(coef_penalizer=.1)
kmf = KaplanMeierFitter()
models = [aaf]

# logistic function
def sigmoid(p,x):
    x0, k = p
    y = 1 / (1 + np.exp(-k*(x-x0)))
    return y

# squared error
def residuals(p,x,y):
    return y - sigmoid(p,x)

# prediction
def prediction_smooth(model, data, i, drop=True, display=True):
    """Returns a decreasing survival function.
    Logisic function fitted to the predicted survival function returned by the mod
el."""
    if drop:
        data_bis = data.drop([i])
    else:
```

```
        data_bis = data
    try:
        model.fit(data_bis, duration_col=duration_col, event_col=event_col, show_p
rogress=True)
    except Exception:
        print("There is a problem")
    axis = plt.subplot(111)
    coord = model.predict_survival_function(data.iloc[i:i+1,2:])
    if display:
        coord.plot(ax=axis)
    coord = coord.reset_index().values
    predicted_i = np.argmin(coord[:,1]>max(0.05, np.min(coord[:,1])))
    x = coord[:predicted_i,0]
    y = coord[:predicted_i,1]
    p_guess = (coord[np.argmin(coord[:,1]>0.5), 0], -1.0)
    p = scipy.optimize.leastsq(func=residuals, x0=p_guess, args=(x,y), full_output
=1)[0]
    if display:
        xp = np.linspace(0, np.max(coord[:,0]), np.max(coord[:,0])*10)
        pxp = sigmoid(p,xp)
        axis.plot(xp, pxp, '-', label = "log")
        kmf.fit(np.array([data[duration_col][i]]), np.array([data[event_col][i]]))
        kmf.plot(ax=axis, linewidth=5.0)
        plt.ylabel('Probability of survival')
        plt.xlabel('Time (days)')
        plt.title('Patient {}'.format(i+1))
        handles, _ = axis.get_legend_handles_labels()
        axis.legend(handles, ["Predicted survival function", "Logistic regression"
, "Real survival function"], loc = 'lower center')
        plt.show()
    return p[0] # median = estimated life expectancy

for m in models:
    for i in range(10):
        prediction_smooth(m, data, i)
```

6.2. References

[1] Cox, D., Reid, N. (1984). Analysis of Survival Data. New York: Routledge.

[2] Efron, B. (1988). Logistic regression, survival analysis, and the Kaplan-Meier curve. Journal of the American statistical Association, 83(402), 414-425.

[3] Carroll, K. J. (2003). On the use and utility of the Weibull model in the analysis of survival data. Controlled clinical trials, 24(6), 682-701.

[4] Aalen, O. (1978). Nonparametric inference for a family of counting processes. The Annals of Statistics, 701-726.

[5] Klein, J. P. (1991). Small sample moments of some estimators of the variance of the Kaplan-Meier and Nelson-Aalen estimators. Scandinavian Journal of Statistics, 333-340.

[6] Perme, M. P., Henderson, R., & Stare, J. (2008). An approach to estimation in relative survival regression. Biostatistics, 10(1), 136-146.

[7] Lin, D. Y., & Wei, L. J. (1989). The robust inference for the Cox proportional hazards model. Journal of the American statistical Association, 84(408), 1074-1078.

[8] Martinussen, T., Vansteelandt, S., Gerster, M., & Hjelmborg, J. V. B. (2011). Estimation of direct effects for survival data by using the Aalen additive hazards model. Journal of the Royal Statistical Society: Series B (Statistical Methodology), 73(5), 773-788

[9] https://cdebrowser.nci.nih.gov/cdebrowserClient/cdeBrowser.html#/search